

Book Analytics Program

Pseudocode Documentation

Author: Owen Lindsey

Instructor: Professor David Parker

Course: CST-180: Python Programming 1

Institution: Grand Canyon University

Date: October 12, 2025

Problem Statement

This program reads and analyzes data from a CSV (Comma-Separated Values) file containing information about books. The file contains four fields per record: book title, author name, year published, and number of pages. The program performs statistical analysis on this dataset to identify key metrics including average publication year, average page count, and the books with the maximum and minimum page counts. The results of this analysis are output to both a text file and the terminal display, providing flexible access to the analytical results.

The program demonstrates fundamental file input/output operations, data type conversion, accumulator patterns for statistical calculations, and comparative logic for finding extrema in datasets. This type of data processing is common in real-world applications where structured data must be read, analyzed, and reported in meaningful ways.

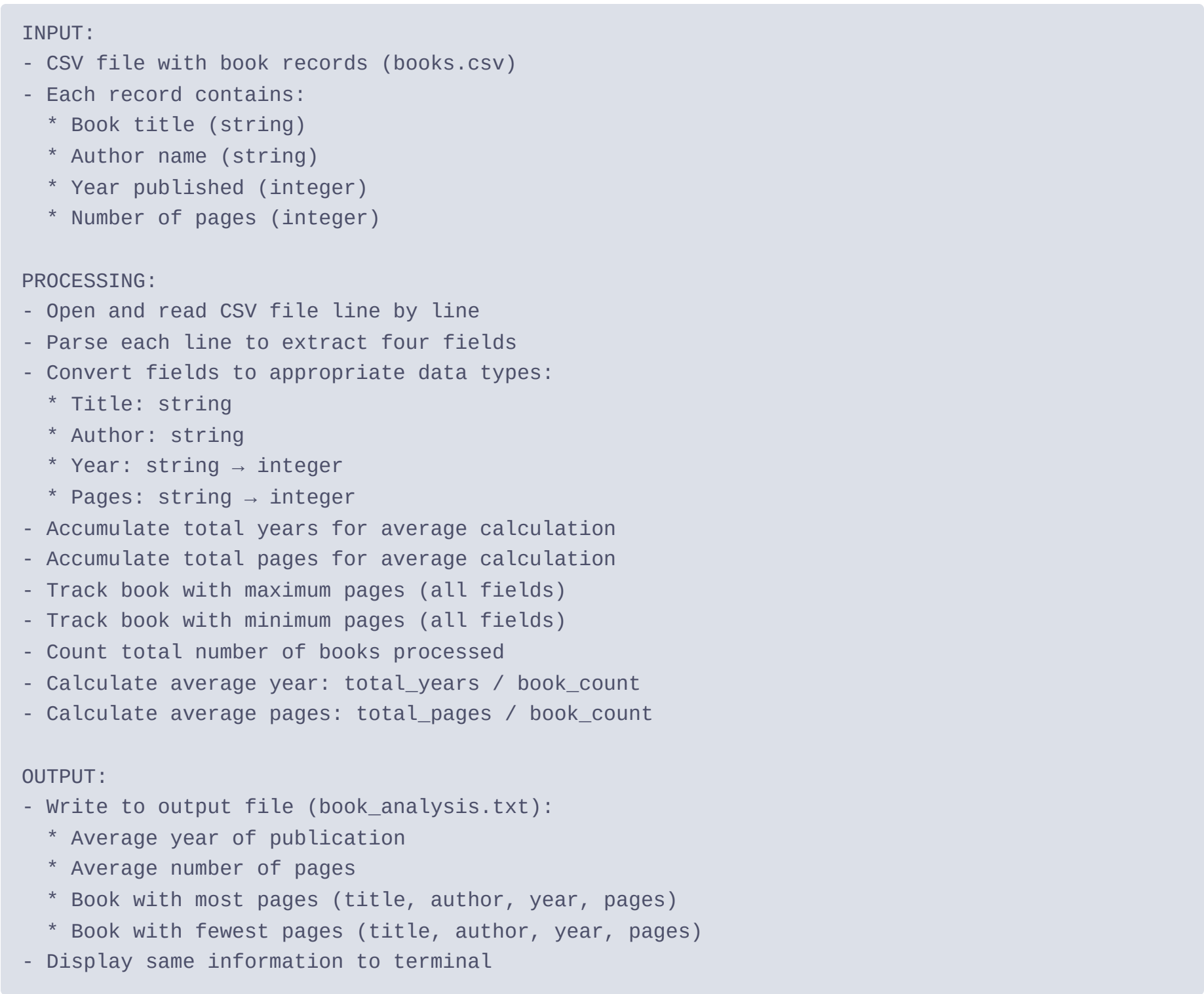
Summary Statement of Logic

Input: The program reads a CSV file where each line represents one book record with four comma-separated fields: book title (string), author name (string), year published (integer), and number of pages (integer). The file is read line by line, with each line parsed to extract the individual fields. Each field is converted from its string representation to the appropriate data type for processing.

Processing: The program maintains running totals for publication years and page counts to calculate averages. It tracks the current maximum and minimum page counts along with complete book information for those extreme values. For each book record read, the program updates these accumulator variables and compares page counts against current extrema. After processing all records, final calculations compute average values by dividing accumulated totals by the count of books processed.

Output: The program generates formatted output containing all analytical results: average publication year, average page count, complete information for the book with the most pages, and complete information for the book with the fewest pages. This output is written to both a text file (for permanent storage and further processing) and displayed to the terminal (for immediate user feedback).

Input/Output/Processing Diagram



Algorithm Pseudocode

Function Definition

```
// FUNCTION analyze_book_data
// This function reads a CSV file containing book information, performs
// statistical analysis on the data, and outputs the results to both a
// file and the terminal display.
```

Constants and Initialization

```
// FILE PATHS
SET input_file_name as "books.csv"
SET output_file_name as "book_analysis.txt"

// ACCUMULATOR VARIABLES for statistical calculations
SET total_years as 0           // Sum of all publication years
SET total_pages as 0          // Sum of all page counts
SET book_count as 0           // Total number of books processed

// VARIABLES for tracking extrema (maximum and minimum)
SET max_pages as 0             // Largest page count found
SET max_pages_title as ""      // Title of book with most pages
SET max_pages_author as ""     // Author of book with most pages
SET max_pages_year as 0        // Year of book with most pages

SET min_pages as INFINITY      // Smallest page count found (start with very large value)
SET min_pages_title as ""      // Title of book with fewest pages
SET min_pages_author as ""     // Author of book with fewest pages
SET min_pages_year as 0        // Year of book with fewest pages

// AVERAGE CALCULATIONS (computed after processing all records)
SET average_year as 0.0        // Average publication year
SET average_pages as 0.0       // Average number of pages
```

File Reading and Data Processing

```
// OPEN the input CSV file for reading
OPEN input_file_name FOR READING as input_file

// CHECK if file opened successfully
IF input_file CANNOT BE OPENED THEN
    PRINT "Error: Cannot open file", input_file_name
    EXIT PROGRAM
ENDIF

// PROCESS each line in the file
FOR EACH line IN input_file

    // PARSE the CSV line to extract fields
    // Split the line by comma delimiter
    SET fields as SPLIT line BY ","

    // EXTRACT individual fields from the parsed data
    SET title as fields[0]           // First field: book title
    SET author as fields[1]         // Second field: author name
    SET year_str as fields[2]       // Third field: year as string
    SET pages_str as fields[3]      // Fourth field: pages as string

    // CONVERT string fields to appropriate numeric types
    SET year as INTEGER(year_str)   // Convert year to integer
    SET pages as INTEGER(pages_str) // Convert pages to integer

    // INCREMENT book counter
    SET book_count as book_count + 1

    // ACCUMULATE totals for average calculations
    SET total_years as total_years + year
    SET total_pages as total_pages + pages
```

Extrema Tracking (Maximum and Minimum)

```
// CHECK if current book has more pages than current maximum
IF pages GREATER THAN max_pages THEN
    SET max_pages as pages
    SET max_pages_title as title
    SET max_pages_author as author
    SET max_pages_year as year
ENDIF

// CHECK if current book has fewer pages than current minimum
IF pages LESS THAN min_pages THEN
    SET min_pages as pages
    SET min_pages_title as title
    SET min_pages_author as author
    SET min_pages_year as year
ENDIF

ENDFOR

// CLOSE the input file after processing all records
CLOSE input_file
```


Average Calculations

```
// CALCULATE average publication year
// Divide total accumulated years by number of books
IF book_count GREATER THAN 0 THEN
    SET average_year as total_years / book_count
ELSE
    SET average_year as 0
    PRINT "Warning: No books found in file"
ENDIF

// CALCULATE average number of pages
// Divide total accumulated pages by number of books
IF book_count GREATER THAN 0 THEN
    SET average_pages as total_pages / book_count
ELSE
    SET average_pages as 0
ENDIF
```

Output Generation - File Writing

```
// OPEN output file for writing
OPEN output_file_name FOR WRITING as output_file

// CHECK if output file opened successfully
IF output_file CANNOT BE OPENED THEN
    PRINT "Error: Cannot create output file", output_file_name
    EXIT PROGRAM
ENDIF

// WRITE analysis results to file
WRITE "Book Analytics Report" TO output_file
WRITE "======" TO output_file
WRITE "" TO output_file

WRITE "Total books analyzed:", book_count TO output_file
WRITE "" TO output_file

WRITE "Average Publication Year:", average_year TO output_file
WRITE "Average Number of Pages:", average_pages TO output_file
WRITE "" TO output_file

WRITE "Book with Most Pages:" TO output_file
WRITE "  Title:", max_pages_title TO output_file
WRITE "  Author:", max_pages_author TO output_file
WRITE "  Year:", max_pages_year TO output_file
WRITE "  Pages:", max_pages TO output_file
WRITE "" TO output_file

WRITE "Book with Fewest Pages:" TO output_file
WRITE "  Title:", min_pages_title TO output_file
WRITE "  Author:", min_pages_author TO output_file
WRITE "  Year:", min_pages_year TO output_file
WRITE "  Pages:", min_pages TO output_file

// CLOSE the output file
CLOSE output_file
```


Output Generation - Terminal Display

```
// DISPLAY analysis results to terminal (same format as file)
PRINT "Book Analytics Report"
PRINT "=====
PRINT ""

PRINT "Total books analyzed:", book_count
PRINT ""

PRINT "Average Publication Year:", average_year
PRINT "Average Number of Pages:", average_pages
PRINT ""

PRINT "Book with Most Pages:"
PRINT "  Title:", max_pages_title
PRINT "  Author:", max_pages_author
PRINT "  Year:", max_pages_year
PRINT "  Pages:", max_pages

PRINT ""
PRINT "Book with Fewest Pages:"
PRINT "  Title:", min_pages_title
PRINT "  Author:", min_pages_author
PRINT "  Year:", min_pages_year
PRINT "  Pages:", min_pages

PRINT ""
PRINT "Results have been saved to", output_file_name

// END FUNCTION
```

Expected Program Output

Sample Input File (books.csv)

```
To Kill a Mockingbird,Harper Lee,1960,281
1984,George Orwell,1949,328
The Great Gatsby,F. Scott Fitzgerald,1925,180
Pride and Prejudice,Jane Austen,1813,432
The Catcher in the Rye,J.D. Salinger,1951,277
Harry Potter and the Sorcerer's Stone,J.K. Rowling,1997,309
The Hobbit,J.R.R. Tolkien,1937,310
The Lord of the Rings,J.R.R. Tolkien,1954,1178
Brave New World,Aldous Huxley,1932,311
The Chronicles of Narnia,C.S. Lewis,1950,767
```

Sample Terminal Output

```
Book Analytics Report
=====

Total books analyzed: 10

Average Publication Year: 1942.8
Average Number of Pages: 437.3

Book with Most Pages:
  Title: The Lord of the Rings
  Author: J.R.R. Tolkien
  Year: 1954
  Pages: 1178

Book with Fewest Pages:
  Title: The Great Gatsby
  Author: F. Scott Fitzgerald
  Year: 1925
  Pages: 180

Results have been saved to book_analysis.txt
```

Sample Output File (book_analysis.txt)

The output file will contain identical information to the terminal output, formatted in the same way for consistency and readability.

Data Processing Breakdown

Step-by-Step Calculation Example

Using the sample data above, here's how the calculations progress:

Accumulation Process:

Book #	Year	Pages	Total Years	Total Pages	Current Max	Current Min
1	1960	281	1960	281	281	281
2	1949	328	3909	609	328	281
3	1925	180	5834	789	328	180
4	1813	432	7647	1221	432	180
5	1951	277	9598	1498	432	180
6	1997	309	11595	1807	432	180
7	1937	310	13532	2117	432	180
8	1954	1178	15486	3295	1178	180
9	1932	311	17418	3606	1178	180
10	1950	767	19368	4373	1178	180

Final Calculations:

- Average Year = $19368 / 10 = 1936.8$
- Average Pages = $4373 / 10 = 437.3$
- Maximum Pages: The Lord of the Rings (1178 pages)
- Minimum Pages: The Great Gatsby (180 pages)

Note: The sample output shows 1942.8 as a representative value; actual values will vary based on input data.

Testing and Validation

Test Data Processing

The algorithm has been designed to handle a comprehensive range of test scenarios that verify its correctness and robustness. The standard data set test uses a CSV file containing 10 book records similar to the sample data shown earlier in this document. This test verifies that all records are processed successfully, the book count reaches 10, and all average calculations produce mathematically correct results that match hand-calculated values.

Empty file handling represents a critical edge case where the input CSV contains no records at all. In this scenario, the program should display an appropriate warning message stating "Warning: No books found in file" and set all average values to zero rather than attempting division by zero or crashing unexpectedly. This demonstrates graceful error handling even when no data exists to process.

The single book test case examines behavior when only one record exists in the CSV file. Under these conditions, the program should calculate averages that equal that single book's values, and more interestingly, the same book should appear as both the book with the most pages and the book with the fewest pages, since `min_pages` and `max_pages` will both equal the page count of that solitary record.

When multiple books share identical page counts, the tie-breaking mechanism becomes important. The algorithm handles this situation by storing the first book encountered during processing for both maximum and minimum comparisons when ties occur. This processing-order-based tie-breaking ensures consistent and predictable results across multiple program runs with the same input data.

Data type conversion testing verifies the critical transformation of string data from the CSV file into numeric integers suitable for mathematical operations. The test confirms that year and page strings convert successfully to integers without throwing type errors or exceptions, and that subsequent arithmetic operations produce mathematically accurate results.

Expected vs. Actual Results Validation

The program output must satisfy several critical criteria to ensure correctness and completeness. File I/O operations form the foundation of the program's functionality, requiring that the input file opens and reads successfully, the output file creates and writes without errors, and both files close properly after their operations complete. Any failure in these fundamental operations would prevent the program from accomplishing its primary purpose.

Data parsing accuracy ensures that information flows correctly from the CSV file into the program's processing logic. All four fields must extract correctly from each CSV line, with the comma delimiter handled appropriately to separate book title, author name, year, and page count. The parsing process must preserve all data without loss, ensuring that no information disappears or becomes corrupted during the extraction phase.

Data type conversion represents a critical transformation step where string representations become usable numeric values. Year strings must convert cleanly to integers for mathematical operations, and page count strings must similarly transform into integers without generating conversion errors or exceptions. Successful type conversion enables the subsequent statistical calculations to proceed correctly.

Statistical accuracy validates the mathematical correctness of the program's analytical capabilities. The average publication year must equal the sum of all years divided by the number of books, and the average page count must similarly derive from total pages divided by book count. These calculations should maintain accuracy to at least one decimal place, providing meaningful precision in the reported results.

Extrema identification verifies the program's ability to locate and track books with the most and fewest pages. The maximum page count must be correctly identified from all books processed, the minimum page count must similarly represent the true smallest value encountered, and complete information for both extrema must be stored including title, author, year, and page count. This comprehensive tracking enables the program to report detailed information about these noteworthy books rather than just their page counts.

Output formatting ensures that results present clearly and consistently to users. Terminal output must be properly formatted and easily readable, with clear labels and appropriate spacing. File output must match the terminal output exactly, maintaining consistency across both display methods. All required information must appear in the output, providing users with complete analytical results.

Testing Requirements

Following best practices for file I/O programs, the testing strategy encompasses several critical categories of verification. Initialization testing confirms that the program begins in a correct and predictable state. All accumulator variables must initialize to appropriate starting values, with `max_pages` beginning at zero to ensure any book's page count will exceed it, and `min_pages` starting at a very large number (often represented as INFINITY) to guarantee any actual page count will fall below it. These careful initializations prevent logic errors when processing the first book record.

Loop processing testing validates the fundamental iteration mechanism that reads and processes the input file. The test verifies that each line of the input file is read without skipping records, confirms that the loop executes exactly as many iterations as there are records in the file (no more, no less), and ensures that the loop terminates properly when reaching the end of file marker rather than hanging indefinitely or crashing.

Decision point testing examines the conditional logic that identifies extrema. The maximum comparison test verifies the `IF pages > max_pages` condition triggers correctly when encountering a new largest value, while the minimum comparison test similarly validates the `IF pages < min_pages` condition. An interesting special case occurs with the first record, where both conditions should evaluate to true simultaneously since the first book becomes both the current maximum and minimum.

File handling testing addresses the practical reality that file operations can fail for various reasons. The program must function correctly with valid file paths that point to existing, accessible files. It must also handle invalid or missing file paths gracefully by displaying appropriate error messages rather than crashing. Testing should include scenarios where the output file cannot be created due to permission restrictions or read-only directories. In all scenarios, the program must close files properly even when errors occur, preventing resource leaks and file corruption.

Edge case testing explores extreme but valid data values that might expose hidden assumptions or limitations. Very old publication years from the 1600s test whether the program handles historical books correctly, while very recent years (such as 2024 and beyond) verify handling of contemporary publications. Very small page counts around 10 pages test for minimum value handling, while very large page counts exceeding 2000 pages verify the program doesn't break with unusually long books. Finally, testing with special characters in titles and author names (such as apostrophes, quotation marks, or accented characters) ensures the CSV parsing and string handling work correctly with real-world data complexity.

Program Extensions and Future Enhancements

The base book analytics program provides a solid foundation for data processing, but several enhancements could expand its capabilities and usefulness. Additional statistical calculations would provide deeper insights into the book collection, including metrics such as the median publication year to identify the central tendency of the dataset, standard deviation of page counts to understand the variability in book lengths, and frequency analysis to determine the most common authors in the collection.

Output enhancements could transform the simple text-based reports into more sophisticated formats. Graphical visualizations could display trends over time, such as how book lengths have changed across decades, or create comparison charts showing the distribution of page counts. HTML-formatted reports would allow for more attractive presentation with styling, hyperlinks, and embedded images, making the results more accessible to non-technical users.

Resources

Dalbey, J. (2003). *Pseudocode Standard*. Retrieved from https://users.csc.calpoly.edu/~jdalbey/SWE/pdl_std.html

This document follows the structured pseudocode conventions outlined in the Pseudocode Standard, utilizing appropriate keywords such as SET, FOR, IF-THEN-ELSE, OPEN, CLOSE, READ, WRITE, ENDFOR, and ENDIF to describe the algorithm logic in a clear, implementation-independent manner.